PYTHON ADVENTURE GAME
GitHub Copilot Assistance Report

Project: Assesment-PythonAdventureGame

PROJECT SUMMARY
A replayable command-line quest in which the player chooses between a dark forest and a mysterious cave, makes follow-up decisions, and seeks treasure.

HOW COPILOT ASSISTED
Copilot-assisted development helped shape the branching story, separate domain rules from CLI interaction, define the application port, and draft automated tests. Suggestions were reviewed against the requirements and verified by running the test suite; the implementation remains dependency-free and uses a lightweight Clean Architecture structure.

IMPLEMENTATION PLAN
1. Put outcomes, choice models, and pure path rules in the domain layer.
2. Define a GameIO protocol in the application layer.
3. Coordinate rounds, paths, and replay in a game application use case.
4. Implement terminal prompts and validation in a console adapter.
5. Wire the adapter and application in the root entry-point module.

CHALLENGES AND RESOLUTIONS
Branching scenarios can be difficult to test when story rules and terminal input/output are mixed together. The domain now decides outcomes without calling the CLI, while the application depends on a GameIO protocol and the console adapter owns input validation. A fake adapter exercises the use case without manual keyboard input.

ENHANCEMENTS TO THE BASIC OUTLINE
- Added a second decision in each route so choices have meaningful outcomes.
- Added numbered and text-based selections, plus invalid-choice recovery.
- Added a default name for blank input and replay after wins or losses.
- Introduced domain, application, and adapter boundaries with dependency direction pointing inward to the application/domain.
- Added automated tests for pure rules, both winning paths, and replay.
- Kept the project standard-library-only for simple setup and submission.

VALIDATION AND RUNNING
The project is tested with: python -m unittest -v
Start the game with: python adventure_game.py
The automated suite covers domain rules, application flows, and CLI input.

Copilot was used as a development assistant; the player-facing story and final code were reviewed and verified by running the project’s tests.